

Java Backend Architecture

Interview Series

Chapter 16.7

Docker Security

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

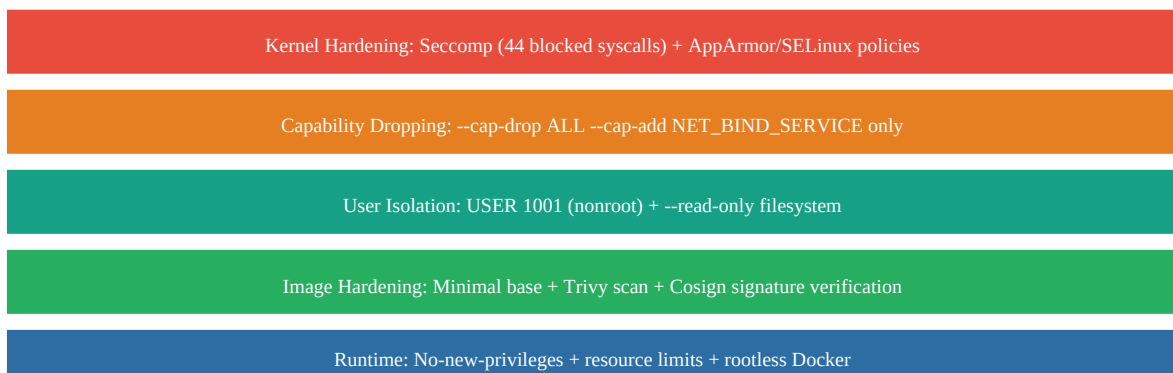
Chapter 16.7 – Docker Security

Docker Security Overview

Container security requires a defense-in-depth approach across multiple layers: image hardening, runtime restrictions, kernel-level controls, and supply chain security. Each layer independently reduces attack surface.

Security Layers

Docker Security Layers – Defense in Depth



Non-Root User

```
# In Dockerfile RUN groupadd --gid 1001 appgroup && \ useradd --uid 1001 --gid appgroup  
--no-create-home --shell /sbin/nologin appuser COPY --chown=appuser:appgroup --from=builder  
/app/app.jar /app/ USER appuser # Runtime enforcement (Kubernetes) securityContext:  
runAsNonRoot: true runAsUser: 1001 allowPrivilegeEscalation: false
```

Read-Only Filesystem

```
# Run container with read-only root filesystem docker run --read-only \ --tmpfs  
/tmp:rw,noexec,nosuid,size=100m \ --tmpfs /app/logs:rw,size=200m \ myapp:latest # In  
docker-compose.yml services: app: read_only: true tmpfs: - /tmp:rw,noexec,nosuid,size=100m
```

Linux Capabilities

Docker grants containers a reduced but still broad set of Linux capabilities by default. Best practice: drop ALL capabilities and add only those actually needed:

```
# Drop all, add only needed docker run --cap-drop ALL \ --cap-add NET_BIND_SERVICE \  
--security-opt no-new-privileges \ myapp:latest # Common capabilities needed by Java apps: #  
NET_BIND_SERVICE: bind port < 1024 (use port 8080+ to avoid this) # SYS_PTRACE: attach debugger  
(dev only) # Most Spring Boot apps need: none (run on port 8080+)
```

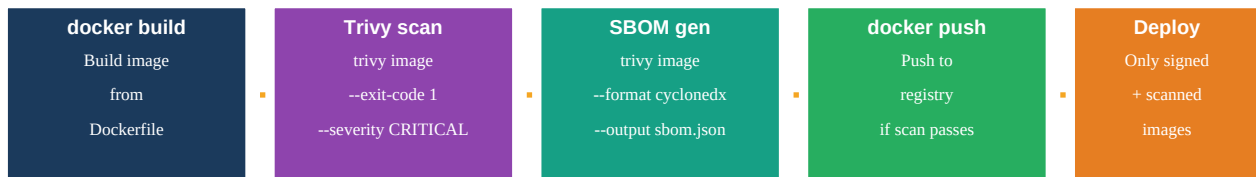
Seccomp Profiles

Seccomp (secure computing mode) filters which Linux syscalls a container can invoke. Docker's default seccomp profile blocks 44+ dangerous syscalls including: `ptrace`, `kexec_load`, `mount`, `pivot_root`, `swapon`, `syslog` (unconfined processes).

```
# Use default seccomp profile (applied automatically) docker run myapp # default profile  
applied # Use custom seccomp profile docker run --security-opt seccomp=/path/to/profile.json  
myapp # Disable seccomp (not recommended) docker run --security-opt seccomp=unconfined myapp #  
Kubernetes: use SeccompProfile securityContext: seccompProfile: type: RuntimeDefault
```

Image Scanning

Trivy Scan Pipeline in CI/CD



Fail build on CRITICAL/HIGH CVEs before push

```
# Trivy - comprehensive scanner trivy image myapp:latest trivy image --severity CRITICAL,HIGH
--exit-code 1 myapp:latest trivy image --format cyclonedx --output sbom.json myapp:latest #
Docker Scout docker scout cves myapp:latest docker scout recommendations myapp:latest # Snky
snky container test myapp:latest --severity-threshold=high # CI/CD GitHub Actions - uses:
aquasecurity/trivy-action@master with: image-ref: myapp:${{ github.sha }} severity:
CRITICAL,HIGH exit-code: 1
```

Cosign Image Signing

Cosign Image Signing + Verification



Keyless signing (Sigstore): identity via OIDC (GitHub Actions / Google / Microsoft)

```
cosign sign --keyless myimage:tag => logged in Rekor transparency log
```

Policy Enforcement: Kyverno or OPA Gatekeeper verify signature before pod start

```
# Generate keys cosign generate-key-pair # Sign image after build cosign sign --key cosign.key
myregistry/myapp:1.0@sha256:... # Keyless signing (Sigstore - GitHub Actions) cosign sign
--keyless myregistry/myapp:1.0 # Verify cosign verify --key cosign.pub myregistry/myapp:1.0 #
Attach SBOM cosign attach sbom --sbom sbom.json myregistry/myapp:1.0
```

Security Best Practices Summary

Practice	Command / Config	Threat Mitigated
Non-root USER	USER 1001 in Dockerfile	Privilege escalation
Read-only FS	--read-only + tmpfs	Persistence after exploit
Cap drop ALL	--cap-drop ALL --cap-add ...	Kernel exploit surface
No new privs	--security-opt no-new-privileges	setuid/setgid abuse
Seccomp	Default or custom profile	Syscall-based attacks
Image scan	Trivy --exit-code 1	Known CVE exploitation
Sign images	cosign sign/verify	Supply chain tampering
Resource limits	--memory --cpus	DoS / fork bombs
Rootless Docker	dockerd-rootless install	Daemon compromise

Interview Q&A; – Docker Security

Q1. What are the most important Docker security best practices?

Key practices: (1) Run as non-root USER — reduces blast radius of container exploits. (2) Use minimal base images (distroless) — fewer packages = fewer CVEs. (3) Read-only filesystem (--read-only + tmpfs for writable paths) — prevents persistence after exploit. (4) Drop ALL capabilities, add only needed. (5) Set resource limits (--memory, --cpus) — prevent DoS. (6) Scan images with Trivy/Snyk in CI/CD. (7) Sign images with Cosign. (8) Never mount docker.sock in containers. (9) Use --security-opt no-new-privileges. (10) Keep base images updated.

Q2. What are Linux capabilities and why should you drop them in containers?

Linux capabilities break the root/non-root privilege model into fine-grained permissions. Docker grants containers ~14 capabilities by default (NET_BIND_SERVICE, CHOWN, SETUID, etc.) even for non-root users. Each capability is a potential attack vector. --cap-drop ALL removes all; then --cap-add adds back only needed ones. For Spring Boot on port 8080+: no capabilities needed. This follows the principle of least privilege — an attacker who exploits the container cannot use capabilities they don't have.

Q3. What is seccomp and how does Docker use it by default?

Seccomp (secure computing) is a Linux kernel feature that restricts which syscalls a process can invoke. Docker applies a default seccomp profile that blocks ~44 syscalls including: ptrace (attach debugger to other processes), kexec_load (load new kernel), mount (mount filesystems), clock_settime (change system clock), add_key/request_key (kernel keyring), and many obscure syscalls used in privilege escalation. If a blocked syscall is invoked, the process receives EPERM or is killed. Custom profiles can be more or less restrictive.

Q4. What is the difference between Trivy, Snyk, and Docker Scout?

Trivy (Aqua Security, open source): comprehensive scanner for OS packages, language dependencies, IaC, secrets, SBOM. Fast, offline capable, widely used in CI/CD. Snyk Container (commercial with free tier): developer-focused, integrates with IDE/Git, provides fix guidance and base image upgrade recommendations. Docker Scout (Docker): built into Docker CLI, integrates with Docker Hub, provides SBOM and policy enforcement. All scan against CVE databases (NVD, GHSA, distro advisories). Trivy is most popular for CI/CD integration due to being open source and comprehensive.

Q5. What is Cosign and how does it improve supply chain security?

Cosign (Sigstore project) signs and verifies container image signatures stored as OCI artifacts in the same registry as the image. Sign: cosign sign --key cosign.key myimage@sha256:...; stores signature as myimage:sha256-abc123.sig in registry. Verify: cosign verify --key cosign.pub myimage — checks signature validity. Keyless mode uses OIDC identity (GitHub Actions, Google, Microsoft) to sign without managing private keys — signatures are logged in Rekor (public transparency log). Policy engines (Kyverno, OPA Gatekeeper) can require valid signatures before deploying.

Q6. What is Docker rootless mode and when should you use it?

Rootless Docker runs the entire Docker stack (daemon + containers) as a non-root user using user namespaces. UID 0 inside the container maps to the actual user's UID on the host. If an attacker escapes the container, they only have the host user's privileges (not root). Install: dockerd-rootless-setuptool.sh install. Limitations: limited networking (no host network on Linux, no privileged ports < 1024 without sysctl tuning), some storage drivers unsupported, no Swarm. Use in: shared CI/CD systems, multi-user workstations, environments with strict no-root policy.

Q7. What is the --security-opt no-new-privileges flag?

`--security-opt no-new-privileges` prevents a process in the container from gaining additional privileges via `execve()` calls — specifically blocks `setuid`, `setgid` binaries, and Linux capabilities granted by file attributes. Without this flag, a non-root process could execute a `setuid` binary (like `su`, `sudo`) and gain root. With `no-new-privileges`, `execve()` cannot increase the process's capabilities. This is separate from `USER` — even if running as root, `no-new-privileges` prevents privilege escalation to capabilities above what the process started with.

Q8. How do you implement a read-only container filesystem?

`docker run --read-only` makes the container's root filesystem read-only (OverlayFS upperdir is also read-only). Applications that write to the filesystem get EROFS (Read-only file system) errors. Solution: mount tmpfs for writable paths: `--tmpfs /tmp:rw,noexec,nosuid,size=100m` and `--tmpfs /app/logs:rw,size=200m`. For Spring Boot: ensure logging config writes to `/tmp/logs` or uses a volume. Benefits: prevents attacker from writing malware, modifying executables, or persisting backdoors. In Kubernetes: `readOnlyRootFilesystem: true` in `securityContext`.

Q9. What CVE severity levels should fail a CI/CD build?

Industry standard: fail builds on CRITICAL and HIGH severity CVEs. MEDIUM: consider failing for new projects or highly regulated environments. LOW: informational only; acceptable to allow. Configuration: `trivy image --severity CRITICAL,HIGH --exit-code 1`. Important nuances: (1) Not all CRITICAL CVEs affect your specific use case (package may be installed but function not called). (2) Check if a fix is available — block only when patch exists. (3) Use ignore files (`.trivyignore`) for accepted risks with documented justification. (4) Time-box: fix within SLA (e.g., 24h for CRITICAL, 7 days for HIGH).

Q10. What is AppArmor and how does Docker use it?

AppArmor (Application Armor) is a Linux MAC (Mandatory Access Control) system that confines programs to a set of allowed resources via profiles. Docker ships a default AppArmor profile (`docker-default`) that is applied to containers. It restricts: mounting filesystems, reading/writing certain `/proc` paths, loading kernel modules, altering network settings. Custom profiles: `docker run --security-opt apparmor=myprofile myapp`. SELinux serves a similar purpose on RHEL/Fedora systems. Both provide kernel-level enforcement that survives container escape to the user space.

Q11. Explain Docker Content Trust (DCT) and its limitations.

Docker Content Trust (`DOCKER_CONTENT_TRUST=1`) uses Notary (TUF framework) to sign and verify image tags. Publishers sign with a private key; clients verify before pulling. Prevents pulling unsigned or tampered images. Limitations: only applies to tags (not digests), requires key management (delegation keys, root keys), Notary server infrastructure needed, limited to Docker Hub natively. Modern replacement: Cosign + Sigstore is preferred — simpler key management (or keyless), OCI-native storage in registry, better tooling integration. DCT is being deprecated in favor of Cosign-based solutions.

Q12. What are the risks of running privileged containers?

`docker run --privileged` disables ALL security restrictions: all capabilities granted, seccomp disabled, AppArmor disabled, device access unrestricted, can mount any host filesystem, can load kernel modules, can modify kernel parameters. Effectively equivalent to root on the host. Risks: container escape trivially achieves full host compromise. Legitimate uses (rare): Docker-in-Docker (DinD) for CI/CD (prefer Kaniko/Buildah instead), device management containers, specific security tools. If you think you need `--privileged`, question whether you really do — usually a specific capability (`--cap-add SYS_PTRACE`) is sufficient.

Q13. What is a Software Bill of Materials (SBOM) and why does it matter for containers?

An SBOM is a formal list of all components in a software artifact — OS packages, language dependencies, licenses, versions, and vulnerabilities. For containers: `trivy image --format cyclonedx --output sbom.json`

myapp generates an SBOM in CycloneDX or SPDX format. Benefits: (1) Know exactly what's in every deployed image. (2) Quickly assess impact of new CVEs (log4shell, Spring4Shell) by querying SBOM. (3) License compliance. (4) Regulatory requirements (US Executive Order 14028). (5) Attestation for supply chain security. Store SBOM as OCI artifact: `cosign attach sbom --sbom sbom.json myimage`.

Q14. How do you handle secrets in containers without Docker Swarm?

Options in order of preference: (1) External secrets manager at startup: app reads from Vault/AWS Secrets Manager at runtime; secret is in memory only. (2) Kubernetes Secrets with External Secrets Operator (syncs from Vault/AWS SM to K8s Secrets, mounted as tmpfs files). (3) Bind mount file at runtime: `docker run -v /run/secrets/db_pass:/run/secrets/db_pass:ro` (file never in image layer). (4) Environment variables (least preferred — visible in `docker inspect`, `ps env`). Never: ARG/ENV in Dockerfile, secrets in image layers, secrets in version control.

Q15. What is the impact of the Docker socket attack surface?

`/var/run/docker.sock` access = root equivalent on host. If mounted in a container (`-v /var/run/docker.sock:/var/run/docker.sock`), the container can: start privileged containers, mount host filesystem via `docker run`, exec into any container on the host, extract secrets from environment variables. Never mount `docker.sock` unless absolutely required (Portainer, legacy CI/CD). Alternatives for CI/CD builds: Kaniko (build without Docker daemon), Buildah (rootless builds), GitHub Actions native build. For monitoring: use metrics API (TCP with TLS) instead of socket.